WR CLICKER

Jeu de type Clicker — Application Web PHP/MySQL

Cahier des Charges Fonctionnel · BTS SIO SLAM · 30/03/2026

Auteurs : Walid – Rémi · Version 1.2

1. Présentation générale du projet

1.1 Contexte

WR Clicker est un jeu web de type « clicker » (ou « idle game ») développé dans le cadre d'un projet BTS SIO SLAM. Le joueur accumule des kilomètres en cliquant sur une voiture et en achetant des améliorations progressives (bonus). L'objectif est d'atteindre le meilleur score possible et de figurer dans le classement général des joueurs.

1.2 Public cible

Le jeu est tout public et vise aussi bien les joueurs occasionnels que réguliers, sans prérequis technique particulier. L'accès se fait via un navigateur web (bureau ou mobile).

1.3 Périmètre fonctionnel

L'application couvre les fonctionnalités suivantes :

- Gestion des comptes joueurs : inscription, connexion, modification, suppression.
- Moteur de jeu : clic, production passive, achat de bonus.
- Sauvegarde de progression et de score en base de données.
- Classement des 5 meilleurs scores enregistrés.
- Page d'aide et guide du joueur.
- Gestion des erreurs avec page dédiée.

2. Architecture technique

2.1 Modèle Client-Serveur

L'application repose sur une architecture client-serveur classique. Le navigateur exécute la logique de jeu en JavaScript (clics, bonus, production passive, affichage du score en temps réel). Le serveur, en PHP, prend en charge toute la persistance : authentification, sauvegarde, classement.

2.2 Stack technologique

Serveur	PHP (architecture MVC simplifiée : vues / traitements / fonctions)
Base de données	MySQL — base wr_clicker, hébergée en local (localhost)
Client	HTML5, CSS3 (variables CSS, Flexbox, Grid), JavaScript ES6
Déploiement	Serveur local Laragon

2.3 Structure des fichiers

Le projet est organisé selon les couches suivantes :

- Couche Modèle — fonctions.php : toutes les fonctions d'accès à la base (INSERT, UPDATE, SELECT, DELETE).
- Couche Vue — vue_*.php, index.php, jeu.php, aideJeu.html : pages affichées à l'utilisateur.
- Couche Traitement — traitement*.php : contrôle des saisies, appels aux fonctions, redirections.
- Ressources — style.css, styleAide.css, script.js : styles et logique côté client.
- Configuration — param_connexion.php : paramètres de connexion à la base.

2.4 Base de données

La base wr_clicker contient trois tables :

Table joueur — stocke les comptes et la progression persistée :

Id_Joueur	INT, clé primaire, auto-incrémentée
Email_utilisateur	CHAR(50) — adresse e-mail unique du joueur
nickname	CHAR(50) — pseudonyme unique
mdp_utilisateur	CHAR(50) — mot de passe (stocké en clair, voir §5 Contraintes)
score	INT — meilleur score enregistré (High Score)
dernierbonus	INT — dernier bonus débloqué (pour restaurer l'image de la voiture)
croissance	INT — production passive au moment de la sauvegarde
croissance2	INT — multiplicateur de clic au moment de la sauvegarde

Table bonus — référentiel des 5 bonus disponibles :

Id_Bonus	INT, clé primaire, auto-incrémentée
nom_bonus	INT — identifiant numérique du bonus

Table appartenir — association Joueur ↔ Bonus avec prix courant :

Id_Joueur	INT, clé étrangère → joueur.Id_Joueur
Id_Bonus	INT, clé étrangère → bonus.Id_Bonus
prix_bonus	INT — prix actuel du bonus pour ce joueur (augmente à chaque achat)

3. Description fonctionnelle détaillée

3.1 Gestion des comptes joueurs

Inscription (vue_formulaire.php → traitement.php → fonctions.php : insert_membre)

- Le joueur saisit un pseudonyme (nickname), une adresse e-mail et un mot de passe.
- Le champ nickname est vérifié côté serveur (non vide). Les champs e-mail et mot de passe sont requis via HTML5.
- La fonction insert_membre() exécute un INSERT INTO joueur. En cas de succès (1 ligne insérée), l'utilisateur est redirigé vers vue_confirmation_inscription.php. En cas d'échec, vers vue_erreur.php.

Connexion (vue_connexion.php → traitement_connexion.php → fonctions.php : connexion)

- Le joueur saisit son pseudonyme et son mot de passe.
- La fonction connexion() exécute un SELECT * FROM joueur WHERE nickname=... AND mdp_utilisateur=.... Elle retourne le tableau complet du joueur si les identifiants correspondent, ou false sinon.
- En cas de succès, une session PHP est démarrée ($_SESSION['nickname'] et $_SESSION['Id_Joueur'] sont stockés) et le joueur est redirigé vers vue_confirmation.php, qui le redirige automatiquement vers jeu.php après 3 secondes.
- En cas d'échec, redirection vers vue_erreur.php avec le message « Utilisateur inexistant ».

Modification / Suppression d'un compte (vue_cherche_id.php → vue_update.php → traitement_update.php)

- La recherche par ID (obtenir_par_id()) affiche les informations du joueur dans un formulaire pré-rempli.
- Le bouton « Supprimer » soumet le formulaire vers traitement_update.php avec le bouton « Supprimer », déclenchant la fonction supprimer_membre() (DELETE FROM joueur WHERE Id_Joueur=...).
- Note : le bouton « Enregistrer » est présent dans le code (traitement_update.php gère les deux cas via $_POST['submit']), mais n'est pas rendu visible dans vue_update.php dans la version actuelle.

3.2 Moteur de jeu (jeu.php + script.js)

L'accès à jeu.php est protégé : si la session ne contient pas de nickname, l'utilisateur est redirigé vers vue_erreur.php. La page charge les données sauvegardées du joueur (score, bonus, progression) depuis la base via PHP, puis les injecte dans des variables JavaScript pour initialiser la session de jeu.

Mécanique de clic :

- La fonction clic() incrémente le score de croissance2 × 1 km à chaque clic sur la voiture.
- croissance2 est le multiplicateur de clic (valeur par défaut : 1, doublée par bonus2, ×10 par bonus4).

Production passive :

- La fonction passif() démarre un setInterval toutes les secondes qui ajoute croissance km au score automatiquement.
- Elle ne s'active qu'une seule fois (garde bool gain) pour éviter les doublons.
- Elle est déclenchée automatiquement au chargement si croissance > 0 (progression restaurée).

Les 5 bonus disponibles :

Bonus	Coût initial	Effet	Multiplicateur de prix	Type d'effet
Bonus 1	25 km	+3 km/s à la production passive (croissance += 3)	×2	Passif
Bonus 2	150 km	Double les km par clic (croissance2 × 2)	×4	Actif
Bonus 3	1 000 km	Double la production passive (croissance × 2)	×5	Passif
Bonus 4	5 000 km	Multiplie par 10 les km par clic (croissance2 × 10)	×12	Actif
Bonus 5	15 000 km	Multiplie par 10 la production passive (croissance × 10)	×30	Passif

À chaque achat de bonus, si ce bonus est le plus avancé jamais acheté (dernierbonus), l'image de la voiture change et un son spécifique est joué (audio/bonusN.mp3).

3.3 Sauvegarde de progression (traitement_sauvegarde.php)

Le bouton « Sauvegarder » soumet un formulaire masqué (POST) contenant l'état complet de la session : score, prix actuels des 5 bonus, croissance, croissance2 et dernierbonus.

Le serveur effectue les opérations suivantes :

- Mise à jour du score dans joueur via insert_sauvegarde() (UPDATE joueur SET score=... WHERE nickname=...).

- Sauvegarde de chaque bonus via sauvegarder_bonus() : INSERT INTO appartenir … ON DUPLICATE KEY UPDATE prix_bonus.
- Sauvegarde de la progression via sauvegarder_progression() : UPDATE joueur SET croissance, croissance2.
- Sauvegarde du dernier bonus débloqué via sauvegarder_dernierbonus().
- En cas de succès, redirection vers vue_confirmation_sauvegarde.php. En cas d'erreur, vers vue_erreur.php.

Note : Le score et les bonus sont restaurés à la prochaine connexion.

3.4 Classement (vue_classement.php)

La page de classement est accessible depuis jeu.php (lien « 🏆 Classement »). Elle affiche les 5 joueurs ayant le meilleur score enregistré, via la requête SELECT * FROM joueur ORDER BY score DESC LIMIT 5. Les colonnes affichées sont : ID Joueur, Nom d'utilisateur, Score.

3.5 Page d'aide (aideJeu.html)

La page d'aide est une page HTML statique accessible depuis jeu.php. Elle explique les règles du jeu, les commandes de base, le détail des 5 bonus (effet et prix de départ), le fonctionnement de la sauvegarde, et donne des conseils stratégiques aux joueurs.

3.6 Gestion des erreurs (vue_erreur.php)

Toutes les erreurs applicatives (champ vide, identifiants incorrects, échec SQL, session absente) redirigent vers vue_erreur.php via un header Location avec un paramètre GET erreur. La page détruit la session active et affiche le message d'erreur. Un lien de retour vers index.php est proposé.

4. Parcours utilisateur

4.1 Nouveau joueur

1. Arrivée sur index.php : présentation du jeu, liens Inscription / Connexion / Recherche.
2. Clic sur « 📝 Inscription » → vue_formulaire.php : saisie du nickname, e-mail, mot de passe.
3. Soumission → traitement.php → insert_membre() → vue_confirmation_inscription.php.
4. Clic sur « 🔑 Connexion » → vue_connexion.php → traitement_connexion.php → session ouverte.
5. Redirection automatique vers jeu.php après 3 secondes.
6. Jeu : clic sur la voiture, achat de bonus, accumulation de score.
7. Clic sur « 💾 Sauvegarder » → traitement_sauvegarde.php → vue_confirmation_sauvegarde.php.

4.2 Joueur existant

1. Connexion via vue_connexion.php avec nickname + mot de passe.

2. La session restaure le score et la progression sauvegardés depuis la base.
3. Le jeu reprend avec le score, les prix de bonus et la production passive précédemment sauvegardés.
4. L'image de la voiture correspond au dernier bonus débloqué.

5. Règles de gestion

RG-01 — Connexion obligatoire	L'accès à jeu.php est conditionné à l'existence de $_SESSION['nickname']. Toute tentative d'accès direct sans session redirige vers vue_erreur.php.
RG-02 — Unicité du compte	Un seul compte par adresse e-mail et par pseudonyme. La contrainte est portée par la base (champs uniques à ajouter) et/ou par le retour d'erreur MySQL lors de l'INSERT (doublon de clé).
RG-03 — Score minimal	L'achat d'un bonus est possible uniquement si score >= prix du bonus. Un alert() JavaScript informe l'utilisateur en cas de score insuffisant.
RG-04 — Prix des bonus croissants	À chaque achat, le prix du bonus est multiplié par son coefficient : ×2 (bonus1), ×4 (bonus2), ×5 (bonus3), ×12 (bonus4), ×30 (bonus5). Le nouveau prix est stocké dans la table appartenir.
RG-05 — Persistance de la progression	Score, prix des bonus, croissance, croissance2 et dernierbonus sont sauvegardés en base et restaurés à chaque connexion. La session de jeu reprend là où elle s'était arrêtée.
RG-06 — Classement	Seuls les 5 meilleurs scores sauvegardés figurent dans le classement. Un score NULL (jamais sauvegardé) n'apparaît pas dans le classement.
RG-07 — Sécurité des sessions	La session est détruite (session_destroy()) en cas d'erreur. Note : les mots de passe sont actuellement stockés en clair — un hachage (password_hash / password_verify) est recommandé pour une mise en production.

6. Contraintes et exigences

6.1 Exigences fonctionnelles

- EF-01 — L'application doit permettre l'inscription, et la connexion à un compte.
- EF-02 — Le score doit s'afficher en temps réel sans rechargement de page.

- EF-03 — Les 5 bonus doivent être achetables et leurs effets appliqués immédiatement.
- EF-04 — La production passive doit démarrer automatiquement si croissance > 0 au chargement.
- EF-05 — La sauvegarde doit persistern en gardant score, bonus, progression et dernierbonus en base.
- EF-06 — Le classement doit afficher les 5 meilleurs scores enregistrés, triés par ordre décroissant.
- EF-07 — L'application doit comporter une page d'aide accessible depuis le jeu.

6.2 Exigences non fonctionnelles

- ENF-01 — Robustesse : toute erreur PHP doit rediriger vers une page d'erreur dédiée, sans afficher de trace technique à l'utilisateur.
- ENF-02 — Évolutivité : la table bonus est conçue pour accueillir de nouveaux bonus sans modification de structure.
- ENF-03 — Performance : la logique de jeu est entièrement côté client (JS) pour éviter les allers-retours serveur pendant la session.